

Actividad 1: Análisis de Código Estático con SAST (*Static Application Security Testing*)

Tema: *Identificación de vulnerabilidades en código fuente*



Objetivo: *Usar SAST para detectar problemas de seguridad en código sin ejecutarlo*

¿Qué es SAST?

SAST (Static Application Security Testing) analiza el código fuente de una aplicación para detectar vulnerabilidades antes de su despliegue.

SEMGREP

Semgrep (Security Enhanced Multi-Grep) es una herramienta de análisis estático de código que permite encontrar errores de seguridad, malas prácticas y vulnerabilidades en el código fuente. Funciona de manera similar a grep, pero con mayor capacidad para comprender la estructura del código y no solo hacer coincidencias de texto.

¿Por qué usar SAST con Semgrep?

- Detección temprana de vulnerabilidades: Permite identificar problemas de seguridad antes de que el código llegue a producción.
- Automatización en CI/CD: Se puede integrar con pipelines de GitHub Actions, GitLab CI/CD, Jenkins, etc.
- Compatible con múltiples lenguajes: JavaScript, Python, Java, Go, C, etc.
- Reglas personalizables: Se pueden definir patrones específicos según los riesgos del proyecto.

Instalar Semgrep en un entorno virtual

Esta opción evita conflictos con los paquetes del sistema y permite gestionar dependencias de forma aislada.

Crear un entorno virtual:

```
python3 -m venv semgrep_env
```

Activar el entorno virtual:

```
source semgrep_env/bin/activate
```

Instalar Semgrep dentro del entorno virtual:

```
pip install semgrep
```

Verificar la instalación:

```
semgrep --version
```

```
(semgrep_env)-(root@kali)-[~]  
# semgrep --version  
1.110.0  
  
(semgrep_env)-(root@kali)-[~]  
#
```

Para **salir del entorno virtual** y volver a la configuración normal del sistema, ejecutar:

```
deactivate
```

Ejemplo de Análisis SAST con Semgrep

NodeGoat <https://github.com/OWASP/NodeGoat> es una aplicación web de código abierto desarrollada por OWASP (*Open Web Application Security Project*) que sirve como plataforma de aprendizaje para la seguridad en aplicaciones **Node.js**.

Su principal objetivo es ayudar a los desarrolladores y profesionales de la seguridad a identificar, comprender y mitigar vulnerabilidades de seguridad en aplicaciones basadas en Node.js y MongoDB.

Características Principales de *NodeGoat*

1. Aplicación Vulnerable Intencionalmente: Contiene fallos de seguridad reales para practicar y aprender. Similar a otras plataformas como OWASP Juice Shop y Damn Vulnerable Web App (DVWA).
2. Simula un Entorno de Producción: Está desarrollada con Node.js, Express.js y MongoDB. Permite a los usuarios comprender cómo se implementan y explotan vulnerabilidades en aplicaciones reales.
3. Vulnerabilidades Presentes en NodeGoat: NodeGoat incluye ejemplos de diversas vulnerabilidades, como:
 - Inyección de MongoDB → (equivalente a SQL Injection en bases de datos NoSQL).
 - Cross-Site Scripting (XSS).
 - Cross-Site Request Forgery (CSRF).
 - Autenticación Débil.
 - Almacenamiento inseguro de credenciales.
 - Ejemplo de uso inseguro de APIs.
4. Ejercicios y Laboratorios Prácticos: OWASP NodeGoat proporciona guías interactivas y retos para que los usuarios exploten y solucionen las vulnerabilidades.
5. Facilidad de Instalación: Se puede ejecutar localmente con Docker, Node.js o en plataformas en la nube como Heroku.

Instalar proyecto **vulnerable** NodeGoat de OWASP.

```
git clone https://github.com/OWASP/NodeGoat.git  
cd NodeGoat
```

Instalar **Semgrep** <https://github.com/semgrep/semgrep>

Semgrep es una herramienta de análisis estático rápida y de código abierto que busca código, encuentra errores y aplica normas de seguridad y estándares de codificación. Semgrep es compatible con más de 30 lenguajes y puede ejecutarse en un IDE, como comprobación previa al compromiso y como parte de flujos de trabajo CI/CD.

```
pip install semgrep  
  
semgrep -version
```

Ejecutar análisis con reglas automáticas de **Semgrep**

```
semgrep --config=auto .
```

--config=auto → Utiliza reglas automáticas recomendadas para detectar vulnerabilidades comunes.
.
→ Escanea todo el código dentro del directorio actual.

Ejemplo de vulnerabilidad detectada:

```
>> javascript.express.security.audit.express-cookie-settings.express-cookie-session-no-secure  
Default session middleware settings: `secure` not set. It ensures the browser only sends the cookie  
over HTTPS.  
Details: https://sg.run/9oKz  
  
78| app.use(session({  
79|   // genid: (req) => {  
80|     //   return genuuid() // use UUIDs for session IDs  
81|   },  
82|   secret: cookieSecret,  
83|   // Both mandatory in Express v4  
84|   saveUninitialized: true,  
85|   resave: true  
86| },  
87| // Fix for A5 - Security MisConfig  
[hid 15 additional lines, adjust with --max-lines-per-finding]
```

```
semgrep --config=auto /var/www/html/
```

```
/var/www/html/login_weak_hash.php  
>>> php.lang.security.injection.tainted-sql-string.tainted-sql-string  
User data flows into this manually-constructed SQL string. User data can be safely inserted into SQL  
strings using prepared statements or an object-relational mapper (ORM). Manually-constructed SQL  
strings is a possible indicator of SQL injection, which could let an attacker steal or manipulate  
data from the database. Instead, use prepared statements ('$mysqli->prepare("INSERT INTO test(id,  
label) VALUES (?, ?)");') or a safe library.  
Details: https://sg.run/LZYG  
  
14| $query = "SELECT password FROM users WHERE username = '$username'";  
  
>>> php.lang.security.injection.printed-request.printed-request  
Printing user input risks cross-site scripting vulnerability. You should use `htmlentities()` when  
showing data to users.  
Details: https://sg.run/QrxEJ  
  
15| Autofix > print(htmlentities("Consulta SQL: " . $query . "<br>"));  
15| print("Consulta SQL: " . $query . "<br>");  
  
/var/www/html/login_weak_mejorado.php  
>>> php.lang.security.injection.tainted-sql-string.tainted-sql-string  
User data flows into this manually-constructed SQL string. User data can be safely inserted into SQL  
strings using prepared statements or an object-relational mapper (ORM). Manually-constructed SQL  
strings is a possible indicator of SQL injection, which could let an attacker steal or manipulate  
data from the database. Instead, use prepared statements ('$mysqli->prepare("INSERT INTO test(id,  
label) VALUES (?, ?)");') or a safe library.  
Details: https://sg.run/LZYG  
  
28| $conn->query("DELETE FROM failed_logins WHERE username = '$username'");
```

Después de ejecutar el comando, Semgrep mostrará una lista de **posibles vulnerabilidades** con detalles como:

- El *archivo* y la *línea* donde se encuentra la vulnerabilidad.
- El *tipo de riesgo detectado* (XSS, Inyección NoSQL, CSRF, etc.).
- *Recomendaciones* para corregir el problema.

Usar Reglas de Seguridad de OWASP

Si quieres aplicar específicamente reglas de seguridad de OWASP, puedes usar:

```
semgrep --config "p/owasp-top-ten" .
```

Esto buscará vulnerabilidades basadas en las 10 amenazas más críticas de OWASP.

Mitigación y Mejores Prácticas

* Evitar concatenación de datos en consultas SQL

Ejemplo inseguro:

```
db.query("SELECT * FROM users WHERE username = '" + user + "'");
```

Ejemplo seguro (uso de parámetros):

```
db.query("SELECT * FROM users WHERE username = ?", [user]);
```

* Automatizar SAST en CI/CD (Integración y Entrega Continua)

CI/CD es un conjunto de prácticas de desarrollo de software que permite la integración automática, pruebas y despliegue de código de manera rápida y segura. Se compone de dos conceptos principales:

CI (Integración Continua - Continuous Integration)

CI se enfoca en **automatizar la integración del código** en un repositorio compartido.

Objetivo:

- Permitir que los desarrolladores suban código frecuentemente.
- Ejecutar pruebas automatizadas para detectar errores lo antes posible.
- Asegurar que cada cambio en el código es estable y funcional.

Ejemplo de CI:

1. Un desarrollador hace un **push** de código a GitHub.
2. *GitHub Actions* (o GitLab CI, Jenkins, etc.) ejecuta pruebas automáticamente.
3. Si las pruebas fallan, el desarrollador recibe una notificación.
4. Si todo está bien, el código se fusiona en la rama principal (main o develop).

Herramientas populares de CI:

- GitHub Actions
- GitLab CI/CD
- Jenkins
- Travis CI
- CircleCI

CD (Entrega y/o Despliegue Continuo - Continuous Delivery / Continuous Deployment)

CD se divide en dos enfoques:

- ☐ **Entrega Continua (Continuous Delivery)**
Se asegura de que el código esté siempre listo para desplegarse, pero el despliegue es **manual**.
Ejemplo: Una actualización está lista en producción, pero un ingeniero revisa y aprueba el despliegue.
- ☐ **Despliegue Continuo (Continuous Deployment)**
Automatiza completamente el despliegue de código a producción **sin intervención manual**.
Ejemplo: Si las pruebas CI pasan, el código se despliega automáticamente en producción.

Herramientas populares de CD:

- Docker
- Kubernetes
- AWS CodeDeploy
- GitHub Actions (para despliegue)
- ArgoCD

¿Por qué es importante CI/CD?

- Reducción de errores → Los problemas se detectan temprano.
- Automatización → Se eliminan procesos manuales, ahorrando tiempo.
- Despliegues rápidos → Facilita la entrega de nuevas funcionalidades.
- Mejora la calidad del software → Pruebas constantes aseguran estabilidad.

CI/CD	Descripción
CI (Integración Continua)	Automatiza la integración y prueba del código en un repositorio compartido.
CD (Entrega Continua)	Código listo para desplegar, pero con aprobación manual.
CD (Despliegue Continuo)	Automatiza completamente el despliegue a producción.

Ejemplo de CI/CD en GitHub Actions

Un pipeline que prueba código y lo despliega automáticamente si todo está bien:

```
name: CI/CD Pipeline  
  
on: [push]
```

```
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Clonar código
        uses: actions/checkout@v4

      - name: Instalar dependencias
        run: npm install

      - name: Ejecutar pruebas
        run: npm test

  deploy:
    needs: build-and-test
    runs-on: ubuntu-latest
    if: success()
    steps:
      - name: Desplegar en producción
        run: echo "Desplegando la aplicación..."
```

1. Crear un workflow en *GitHub Actions*

GitHub Actions usa archivos YAML dentro de `.github/workflows/` para definir los pipelines.

Crea un archivo en tu repositorio:

```
.github/workflows/semgrep.yml
```

Agrega el siguiente contenido para ejecutar **Semgrep** en cada **push** y **pull request**:

```
name: Semgrep SAST Scan

on:
  push:
    branches:
      - main
      - develop
  pull_request:

jobs:
  semgrep:
    runs-on: ubuntu-latest
    steps:
      - name: Clonar el código
        uses: actions/checkout@v4

      - name: Instalar Semgrep
        run: pip install semgrep

      - name: Ejecutar análisis de seguridad
        run: semgrep --config=auto --json --output=semgrep-results.json

      - name: Mostrar resultados en la salida de GitHub Actions
        run: cat semgrep-results.json

      - name: Guardar reporte de Semgrep como artefacto
        uses: actions/upload-artifact@v3
        with:
```

```
name: semgrep-report
path: semgrep-results.json
```

Explicación del Workflow

- on:
 - Ejecuta el análisis en cada push y pull request en las ramas main y develop.
- jobs:
 - Ejecuta Semgrep en un runner ubuntu-latest.
 - Instala Semgrep usando pip install semgrep.
 - Ejecuta el escaneo con reglas automáticas (--config=auto).
 - Guarda los resultados en un archivo JSON (semgrep-results.json).
 - Muestra los resultados en la consola.
 - Guarda el reporte como artefacto, para revisar los hallazgos más tarde.

2. Usar Reglas Personalizadas en Semgrep

Si quieres definir tus propias reglas en Semgrep, sigue estos pasos:

Crea un archivo YAML con reglas personalizadas `semgrep-rules.yaml`

```
rules:
- id: detect-insecure-sql
  patterns:
    - pattern: db.query("SELECT * FROM users WHERE username = " + $X)
  message: "Posible inyección SQL detectada"
  languages: [javascript]
  severity: ERROR
```

Modificar el workflow de GitHub Actions para usar las reglas personalizadas

```
- name: Ejecutar análisis con reglas personalizadas
  run: semgrep --config=semgrep-rules.yaml --json --output=semgrep-results.json
```

3. Integrar Semgrep en CI/CD con GitLab CI

Si usas **GitLab CI/CD**, puedes agregarlo a `.gitlab-ci.yml`:

```
stages:
- security

semgrep-sast:
  image: python:3.10
  stage: security
  before_script:
    - pip install semgrep
  script:
    - semgrep --config=auto --json --output=semgrep-results.json
  artifacts:
    paths:
      - semgrep-results.json
```

Usar reglas personalizadas en Semgrep se pueden definir en un archivo YAML `custom-rules.yaml`:

```
rules:
- id: detect-insecure-sql
  patterns:
    - pattern: db.query("SELECT * FROM users WHERE username = " + $X)
  message: "Posible inyección SQL detectada"
  languages: [javascript]
  severity: ERROR
```

Y ejecutarlas con:

```
semgrep --config=path/to/custom-rules.yaml .
```